

The Generated DC-RS Cheatsheet on APEX-Agents (Investment Banking)

REUSABLE CODE AND TOOL-CALL SNIPPETS

When using the code_execution_server_code_exec tool to run Python, always invoke as python3 -c followed by the code in double quotes. Use semicolons for simple statements on one line; for multi-line code with indentation, embed newlines as \n within the string. Bare code or python (without 3) causes shell interpretation or command-not-found errors.

```
python3 -c "r = 0.0985; fcf5 = [149.1387682, 144.3343349, 150.5161488, 157.7324231, 165.1306757, 172.7043214]; tv = 2596.867898; cash = 182.947; debt = 0; shares = 50.574641; discounted = 0.0;\nfor n in range(1,7):\n    df = 1 / (1 + r) ** (n - 0.5)\n    cf = fcf5[n-1] if n < 6 else fcf5[n-1] + tv\n    discounted += cf * df\n    print('New EV:', discounted)"
```

SOLUTION STRATEGIES FOR RECURRING TASK SHAPES

When an existing DCF model must be updated to midyear convention while preserving all other inputs and assumptions, modify only the timing exponents used in the present-value calculations; do not alter revenues, margins, growth rates, capex, working capital, tax rates, or terminal-value parameters.

Locate the discounting formulas; replace each year-n end-of-year exponent with (n - 0.5); recompute the PV of each cash-flow line and the total enterprise value; divide by shares outstanding to obtain per-share value.

When a financial model task requires finding the value of an input parameter that produces a target output metric (such as a specific IRR, yield, or P/E ratio), implement a Python replica of the model's key calculations and use a numerical search method such as bisection to solve for the parameter when the relation is nonlinear or complex; for direct relations such as P/E target, solve algebraically as earnings = (implied_share_price * shares) / target_pe and compute the required adjustment from the original value.

```
def solve_for_param(target_metric, low=0.0, high=0.2, tol=1e-6):
    for _ in range(50):
        mid = (low + high) / 2
        metric = compute_model_metric(mid)
        if metric < target_metric:
            low = mid
        else:
            high = mid
        if abs(metric - target_metric) < tol: break
    return round(mid, precision)
# For P/E: required_earnings = implied_price * shares / target_pe; delta = required_earnings - original
```

When an LBO model task requires revising the offer premium to CNS holders or the maximum Term Loan B leverage, update only the affected entry valuation (new share price = current * (1 + premium)) and debt amount (TLB drawn = new max); preserve all operating forecasts, margins, working capital, capex, tax, exit multiple, and debt terms; then recalculate entry equity value, sponsor equity, debt repayment schedule, and exit equity to obtain the revised IRR and MoM.

premium = task_premium; max_tlb = task_max_tlb; entry_price = current_price * (1 + premium); tlb = max_tlb; recompute_sources_uses(); run_debt_schedule(); compute_exit_equity(); irr = ...; mom = ...

When a valuation task requires implied equity values from Comparables, Precedents, DCF, and LBO methods, compute each separately using the task-specified inputs (2025E EBITDA for comps and precedents; base DCF model and assumed offer price for DCF and LBO), then calculate the arithmetic mean of the four, round to one decimal place, and deliver all five numbers (four methods + mean) in \$ million.

implied_comps = ...; implied_precedents = ...; implied_dcf = ...; implied_lbo = ...; mean = round((implied_comps + implied_precedents + implied_dcf + implied_lbo)/4, 1); submit "x.x y.y z.z w.w mean m,m"

When an LBO task requires building sensitivity tables for Sponsor IRR under different offer premiums and exit multiples while respecting a TLB cap, vary the premium (to recompute entry price, equity value, and sponsor equity after enforcing the TLB maximum and balancing sources/uses at constant entry EV) and exit multiple (to recompute exit EV and equity); for each combination run the debt schedule and derive IRR; present results in a 2x2 table (rows = premiums, columns = exit multiples) rounded to one decimal place.

For premiums 15.0%, 25.0% and exit multiples 17.5x, 18.5x: compute the four IRR values from the capped-TLB structure and format as table.

When a DCF task requires revising an operating expense assumption such as employee compensation expense, extract the base compensation values from the model, apply the specified reduction (e.g., 50% of a referenced stock-based compensation value), update the NOPAT and Unlevered FCF calculations for the affected years, then recompute the enterprise value and per-share value using the standard DCF discounting and terminal value methods while preserving all other inputs including WACC, terminal growth, and shares.

compensation_reduction = 0.5 * stock_based_comp_value; updated_comp = base_comp - compensation_reduction; updated_nopat = base_nopat - (compensation_reduction * (1 - tax_rate)); updated_fcf = ...; then run standard DCF code with updated_fcf

When a valuation task requires an implied share price from filtered precedents (AUM >= \$90B) combined with adjusted DCF, filter the precedents table to retain only transactions meeting the AUM threshold, extract the mean or median EV/EBITDA multiple, adjust the DCF forward (2025E) total expenses upward by 10% and D&A by 5% to obtain the adjusted EBITDA, compute Implied EV = multiple * adjusted_EBITDA, Implied Market Cap = Implied EV - Net Debt, and share price = Implied Market Cap / FDSO; round all values to 2 decimal places and edit the 'Precedents' tab to insert the share price build starting with the columns EV/EBITDA, 2025E EBITDA, Implied EV, Net Debt, Implied Market Cap, FDSO, share price.

Filter precedents AUM >=90; multiple = mean_or_median; adjusted_ebitda = base_ebitda_with_expense_adj; ev = multiple * adjusted_ebitda; mkt_cap = ev - net_debt; price = round(mkt_cap / fdso, 2); edit 'Precedents' tab with build columns.

When a DCF task requires revising revenue growth rates by business segment while preserving all other inputs, project each revenue segment independently using its specified growth rate to obtain forward revenues, compute total revenue, apply the operating expense and capex percentages of total revenue from the LBO Model-hardcoded tab to derive updated expenses and capex, recalculate NOPAT and unlevered FCF for each year, then recompute enterprise value and implied share price using the standard DCF discounting and terminal value methods (Gordon growth or exit multiple) while preserving WACC, tax rate, shares, and terminal parameters.

rev_seg1 = base_seg1 * (1 + g1)**n; rev_seg2 = base_seg2 * (1 + g2)**n; rev_seg3 = base_seg3 * (1 + g3)**n; total_rev = rev_seg1 + rev_seg2 + rev_seg3; opex = total_rev * pct_opex_from_lbo; capex = total_rev * pct_capex_from_lbo; ...; then run standard DCF with updated FCFs for both terminal methods.

When a DCF task requires the terminal value based on an EV/AUM multiple drawn from a peer set (with specified peers excluded), extract the relevant EV/AUM multiples from the peer data, compute the mean or median multiple, project forward AUM using the task-specified growth relation (such as linear with management fees), compute TV = multiple * projected_AUM, then integrate into the standard DCF discounting of FCFs plus TV to obtain enterprise value, equity value, and per-share value while preserving all other model inputs and rounding to the required precision.

multiple = mean_or_median(ev_aum_from_peers_excluding_specified); projected_aum = ...; tv = multiple * projected_aum; then discount FCFs + TV with standard WACC to get EV and per-share value.

When a DCF task requires simultaneously lowering all revenue growth rates by a fixed number of percentage points and raising employee compensation as a percentage of revenue by a fixed number of percentage points while also applying midyear convention, implement the combined adjustments in a Python replica of the model: subtract the pp shift from each original growth rate to project revenues, add the pp shift to the compensation pct to derive updated opex and NOPAT, compute the resulting FCF series, discount using (n-0.5) exponents, and obtain per-share values separately for Gordon growth and exit-multiple terminal methods while preserving WACC, tax rate, shares, and other inputs.

adjusted_g = base_g - delta_pp; adjusted_comp_pct = base_comp_pct + delta_pp; ... project revs and fcf with adjusted_g and adjusted_comp_pct; then discount with midyear df for both terminal methods to obtain the two share prices.

When a DCF task requires calculating the implied terminal growth rate consistent with a terminal value based on an EV / mgmt fees (or AUM) multiple from a peer set (with specified peers excluded), extract the EV / mgmt fees multiples from the peer data, compute the mean or median multiple, project the terminal mgmt fees using the task-specified growth relation (such as linear with management fees or revenue), compute the target TV = multiple * projected_mgmt_fees, then back out the implied g = WACC - FCF_{n+1} / TV and round the percentage to two decimal places while preserving all other model inputs and assumptions.

multiple = mean_or_median(ev_mgmt_fees_from_peers_excluding_specified); projected_mgmt_fees = ...; tv = multiple * projected_mgmt_fees; implied_g = wacc - fcf_next / tv; round(implied_g * 100, 2)

When a DCF task requires adjusting one or more revenue growth rates (including to achieve a target multi-year CAGR by solving for a specific intermediate year) while extending the projection horizon and preserving other assumptions, implement the changes in a Python replica: solve for the required growth rate(s) using the CAGR formula rearranged for the target year, extend the forecast to the new horizon by copying the final year's assumptions forward, adjust terminal growth if specified, recompute FCF series, discount, and obtain per-share values for Gordon growth and exit-multiple terminal methods while preserving WACC, tax rate, shares, and other inputs; round results to two decimal places.

target_cagr = (end_rev / start_rev) ** (1 / n) - 1; solve for g_specific such that the compound growth over the window equals target_cagr; extend projections by repeating year-n assumptions for year n+1; terminal_g_new = terminal_g + 0.0025; project FCFs; discount and compute share prices for both terminal methods.

When an LBO task requires computing the equity value at a specific projection year after replacing the exit EBITDA multiple with a peer-derived average (such as the mean FY2023 EV/EBITDA from specified peers), update only the exit multiple parameter; preserve all operating forecasts, margins, debt terms, and other assumptions; recompute exit EV as exit EBITDA times the new multiple, subtract net debt at the target year to obtain equity value, and round to the nearest million.

exit_multiple = mean(ev_ebitda_from_peers); exit_ebitda = model_exit_ebitda; exit_ev = exit_ebitda * exit_multiple; equity = exit_ev - net_debt_at_year; round(equity)

When a DCF task requires updating a specific year's revenue growth rate to achieve a target CAGR for R&D (or similar metric) over the projection period while preserving other growth rates, solve for the target year's growth using the CAGR multiplier equation (target_mult = (1 + cagr)**n_years; g = target_mult / known_mult - 1) and implement in a Python replica of the model to recompute FCFs, TV, EV, and per-share value.

cagr = (end_rd / start_rd) ** (1/n) - 1; target_mult = (1 + cagr)**n; known_mult = g1 * g2 * g3 * g5 * g6; g4 = target_mult / known_mult - 1; update 2027 g with g4; recompute DCF

When a DCF task requires adjusting operating expenses (ex R&D) in a specific year to make the period CAGR equal to half a peer's G&A CAGR, solve for the 2028 opex value that satisfies the CAGR equation while preserving other years' opex, then update the model and recompute the DCF share price.

target_cagr = (end_opex / start_opex) ** (1/n) - 1; opex_2028 = opex_2027 * (1 + target_cagr); update model opex_2028; recompute FCF and share price

When an LBO model task requires revising the offer premium to CNS holders or the maximum Term Loan A or Term Loan B leverage, update only the affected entry valuation (new share price = current * (1 + premium)) and debt amount (TLA or TLB drawn = new max, with any remaining funding requirement satisfied through increased equity contribution); preserve all operating forecasts, margins, working capital, capex, tax, exit multiple, and debt terms; then recalculate entry equity value, sponsor equity, debt repayment schedule, and exit equity to obtain the revised equity value at the target year.

max_leverage = task_max_leverage; base_ebitda = model_2023a_ebitda; max_tla = max_leverage * base_ebitda; tla = max_tla; equity_contrib = remaining_funding; recompute_sources_uses(); run_debt_schedule(); compute_exit_equity_at_year(); equity_value = exit_ev - net_debt_at_year; round(equity_value)

When an LBO task requires applying a revenue growth rate derived from the average of specified peers' year-over-year revenue growth, extract the peer YoY growth rates for the relevant year, compute the arithmetic mean as the constant growth rate, apply it to the base year revenue to project forward revenues for the projection period, preserve all other operating assumptions, margins, expenses, capex, working capital, tax, debt terms, and exit multiple; then run the debt schedule and compute the exit year equity value.

peer_growth = [g_peer1, g_peer2, g_peer3]; growth = mean(peer_growth); rev = [rev_base]; for i in range(n_years): rev.append(rev[-1] * (1 + growth)); ... ; exit_ev = ebitda_exit * exit_multiple; equity = exit_ev - net_debt_exit; round(equity)

When a DCF task requires updating the equity risk premium to be the risk-free rate plus 150 basis points and the cost of debt to be the risk-free rate plus 300 basis points, retrieve the specified risk-free rate (5-year or 7-year treasury on the given date), set rf, compute ERP = rf + 0.015, cod = rf + 0.03, update WACC, then compute the implied share prices for each combination of terminal growth rate (1.25% or 1.75%) and rf tenor, reporting the four values rounded to two decimal places.

rf_5y = calendar...("2025-12-12", "5y"); rf_7y = calendar...("2025-12-12", "7y"); for each g in [0.0125, 0.0175] and each rf: update wacc; compute share_price; output the four prices

When the task explicitly states "Don't edit any files, just print your answer back here", implement all model updates and the full DCF calculation exclusively in a Python code replica executed via code_execution_server_code_exec; do not call any sheets edit or modification tools.

In code: set growth = 0.12 for all years; rf_1y = ...; rf_5y = ...; cod = (rf_1y + rf_5y)/2 + 0.01; beta = 1.3; update wacc with new cod and beta; project revs, fcf with constant g; compute share price; print the rounded value.

When a DCF task requires adjusting gross margin and operating expense percentages based on a competitor identified by the lowest EV/Revenue multiple, first locate that competitor in the comparable analysis file, extract its FY2024 gross margin (add 10 percentage points for the update) and the average SG&A as a percentage of revenue across FY2021-FY2024, replace the model's gross margin and opex rates for all projection periods with these values while preserving all other assumptions including growth rates, capex, working capital, tax, WACC components, and terminal parameters, update the risk-free rate to the specified 20-year treasury yield on the given date, then recompute the PV of FCFs (sum of discounted unlevered free cash flows over the projection horizon) using the standard DCF discounting and round to the nearest whole number.

competitor = lowest_ev_rev_from_comps; gm_new = competitor_fy2024_gm + 0.10; opex_new = mean(competitor_sg_a_pcts_fy2021_to_fy2024); rf_new = treasury_20y_on_2025_10_20; update model rates and rf; project updated FCF series; pv_fcf = sum((1/(1+wacc)**n * fcf_n for n); round(pv_fcf)

FORMULAS, DEFINITIONS, AND CONVENTIONS

Midyear convention treats each period's free cash flow as occurring at the midpoint of the year. For annual periods the discount factor for year n therefore uses the fractional exponent (n - 0.5).

DF_n = 1 / (1 + r) ** (n - 0.5)

The final per-share DCF value is the enterprise value (sum of discounted FCFs plus discounted terminal value) minus net debt, divided by the number of shares, then rounded to the nearest cent.

Value_per_share = round((EV - Net_Debt) / Shares , 2)

In LBO models, MoM (Multiple of Money) is exit equity value divided by sponsor equity invested. IRR is the compound annual return solving for the rate that equates sponsor cash flows, often approximated from MoM as (MoM ** (1/ holding_years)) - 1.

MoM = exit_equity / sponsor_equity
IRR_approx = MoM ** (1 / n_years) - 1

Implied Adj. EBITDA Entry Multiple (2024A) is entry enterprise value projected by 2024A Adj. EBITDA.

Implied Entry Multiple = Entry_EV / 2024A_Adj_EBITDA

Implied terminal growth rate g consistent with a target TV is given by g = WACC - FCF_{n+1} / TV.

g = r - FCF_{n+1} / TV

ENVIRONMENT AND SANDBOX FACTS

The code execution environment has Python 3 available and requires the explicit python3 command; the plain python command is not available or leads to shell interpretation.

Invoke as python3 -c "code"; confirm with which python3 if needed.

The calendar_server_calendar tool is for managing calendar events and requires an event_id for read actions; it does not provide access to treasury rates or financial data. Attempts to use it for rates result in "Required: event_id" errors.

calendar_server_calendar with action="read", start_date=..., end_date=... returns error; use only for event management if needed.

PITFALLS, EDGE CASES, AND VERIFICATION CHECKS

After applying midyear convention, confirm that only the discount exponents were changed and that the delivered numeric answer matches the required rounding and delivery format (nearest cent, no extra text).

Verify rounding: final numeric result rounded to exactly two decimal places; confirm no other model inputs were edited.

When the task provides specific assumptions (WACC, terminal growth, exit multiple) that differ from the model's built-in values, override with the task-specified values for the calculation; do not use the spreadsheet's embedded rates.

Confirm WACC = task_WACC, g = task_g, multiple = task_multiple before computing exit values or yields.

When the task specifies the precision for the answer (e.g., to one decimal place or nearest integer), round the final numeric results accordingly and deliver only the numbers in the requested format without extra text; for amounts in millions, report the rounded integer value with unit if required by the task.

final_value = round(computed); submit exactly as the required short reply with dollar amount in millions.

When filtering precedents for AUM threshold and applying specified expense adjustments before computing implied share price, confirm only transactions with AUM >= \$90B are retained, only total expenses (+10%) and D&A (+5%) are increased on the DCF forward figures, the share price build is placed starting in the 'Precedents' tab, and all values are rounded to exactly two decimal places.

Verify AUM filter excludes <\$90B; confirm expense and D&A adjustments only; check build columns in 'Precedents' tab; round all to 2 decimals.

When applying simultaneous revenue-growth and compensation-pct shifts plus midyear discounting, confirm that only the specified growth rates were reduced, only the compensation percentage was increased, the discount exponents use (n-0.5), and each terminal method's per-share result is reported separately to two decimal places in the requested message format.

Verify growth rates reduced by exactly the stated pp, compensation pct increased by exactly the stated pp, midyear exponents used, and output format is two separate USD values rounded to two decimals.

When computing an implied terminal growth rate from a peer-based TV, confirm that only the specified peers were excluded when calculating the mean or median multiple, the mgmt fees (or AUM) projection used the linear relation from the base year, the solved g is reported to exactly two decimal places as a percentage, and no other model inputs or assumptions were altered.

Verify peers excluded match the task list; confirm mgmt fees projection method; check g rounded to two decimals with % sign or per required format.

When matching CAGRs from peer financials (e.g., PWSC R&D 2019-2023) to model projection (2024-2028), confirm the exact periods, the correct expense line (R&D or G&A), and that the opex adjustment is applied only to the specified year and metric before recalculating.

Verify start=2019 end=2023 for peer, model years 2024 to 2028; confirm R&D for growth, G&A for opex; check only 2027 g and 2028 opex changed.

When computing multiple DCF share prices for different terminal growth rates and risk-free rate tenors, confirm that the correct rf (5y vs 7y) is used for each scenario, the ERP and cost of debt are updated accordingly, and the four prices are reported in the order specified by the task (g=1.25% 5y, g=1.25% 7y, g=1.75% 5y, g=1.75% 7y) rounded to two decimal places.

Verify order of output matches task; check each rf tenor matches the scenario; round each price to exactly 2 decimals.

When the task prohibits editing files, verify that no sheets edit calls are made and that the final answer is printed directly as the response without file changes.

Confirm no "edit" or "add_content" actions on sheets; final output is the numeric share price only.